CloudSage — Build Runbook

From Ubuntu 25.10 setup to the first working EC2 optimization flow

0. Development Rule

Multiple AI coding assistants may be used during development. They are not runtime dependencies. After the project works, runtime uses one selected LLM API/provider and one API key.

1. Target Environment

- Ubuntu 25.10
- Git
- Python 3.x
- Docker Engine/Compose
- AWS CLI
- Terraform
- PostgreSQL
- Optional Node.js later for UI

2. Install Base Packages

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git curl wget unzip jq build-essential python3 python3-venv python3-pip
```

3. Verify

```
git --version
python3 --version
pip3 --version
docker --version
aws --version
terraform version
```

Install any missing AWS CLI/Terraform/Docker packages using their current official installation instructions; pin versions in project documentation after installation.

4. Create Repository

```
mkdir -p ~/projects/cloudsage
cd ~/projects/cloudsage
git init
printf '# CloudSage\n' > README.md
```

5. Create Python Environment

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

6. Initial Python Dependencies

```
pip install fastapi uvicorn boto3 pydantic-settings sqlalchemy psycopg[binary] python-dotenv httpx pytest ruff
```

Freeze dependencies only after the first stable environment is confirmed.

```
pip freeze > requirements.txt
```

7. Recommended Repository Layout

```
cloudsage/
├── app/
│   ├── api/
│   ├── collectors/
│   ├── optimizers/
│   ├── ai/
│   ├── terraform/
│   ├── policies/
│   ├── verification/
│   ├── db/
│   └── main.py
├── tests/
├── terraform/
├── scripts/
├── docs/
├── .env.example
├── .gitignore
├── Dockerfile
├── compose.yaml
├── requirements.txt
└── README.md
```

8. AWS Authentication

Prefer an IAM role or short-lived credentials. For local development, configure the AWS CLI using the official supported mechanism. Never commit credentials.

```
aws configure
aws sts get-caller-identity
```

9. Create Read-Only MVP Permissions

The initial role should permit only the read operations required for EC2 metadata, Cost Explorer and CloudWatch. No Terraform apply or destructive permissions are needed for V0/V1.

10. Build the First Collector

Implement an EC2 collector that returns instance ID, type, state, region, launch time, tags and relevant metadata.

11. Add CloudWatch Metrics

Collect at least CPUUtilization and useful network/health metrics. Aggregate 7-day and 30-day windows, including average, P95 and maximum where practical.

12. Add Cost Collection

Use AWS Cost Explorer to retrieve the account/service/time-window cost data available to the project. Document any resource-level attribution limitations.

13. Build Deterministic Candidate Rules

Example MVP rule:
- Resource is running
- Sufficient historical data exists
- Average CPU is below configured threshold
- P95/max are below configured safety thresholds
- No recent health failures
- No active incident/alert flag
→ candidate = possible_rightsizing

14. Build Savings Calculator

Calculate current and proposed costs using deterministic pricing data/assumptions. Never ask the LLM to calculate money. Record the pricing source and timestamp.

15. Build AI Adapter

Create one provider-neutral interface such as analyze_optimization(context). Only one provider implementation is enabled at runtime.

16. Define Structured AI Output

decision
optimization_type
current_configuration
recommended_configuration
estimated_monthly_savings
risk
confidence
reasoning
evidence

concerns
required_validation

17. Add Validation

Reject malformed AI output. Cross-check referenced resource IDs, current values and savings against deterministic backend data.

18. Add PostgreSQL

```
docker run --name cloudsage-postgres -e POSTGRES_PASSWORD=devpassword -e POSTGRES_DB=cloudsage -p 5432:5432 -d postgres:latest
```

For a real deployment, pin the PostgreSQL image version and use secrets rather than the example development password.

19. First API

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Expose an endpoint that starts an EC2 scan and another that returns recommendations.

20. First End-to-End Run

AWS → EC2 collector → CloudWatch → Cost Explorer → candidate rules → context builder → one LLM API → schema validation → deterministic savings calculation → recommendation stored in PostgreSQL → API response

21. Add Terraform Intelligence

```
terraform init
terraform fmt -check
terraform validate
terraform plan -out=tfplan
terraform show -json tfplan > tfplan.json
```

Use Terraform state/JSON output to map AWS IDs to Terraform resources. Do not apply anything yet.

22. Add Recommendation Approval

Store an approval record. The approval endpoint should be separate from recommendation creation.

23. Add Terraform Change Generation

Generate a controlled patch or modify a known Terraform variable/module input. Then run fmt and validate before plan.

24. Add Plan Safety Gate

Block if the plan contains unexpected destroys, unrelated resource changes, or policy violations. The AI can explain the plan, but deterministic policy decides whether the plan is allowed.

25. Add Apply

terraform apply tfplan

Only execute after the required human approval and policy checks.

26. Add Verification

Capture the pre-change baseline, monitor the resource after deployment, and compare health, utilization and cost. Mark the optimization successful, inconclusive or failed.

27. Containerize

docker build -t cloudsage:dev .
docker run --rm -p 8000:8000 --env-file .env cloudsage:dev

28. CI/CD

Add GitHub Actions or Jenkins for linting, unit tests, integration tests, Docker build and Terraform validation. Deployment should not automatically apply production infrastructure.

29. Final MVP Acceptance Test

1. One EC2 resource is discovered.
2. Cost and utilization data are collected.
3. A deterministic rule identifies a candidate.
4. The LLM returns structured reasoning.
5. Savings are independently calculated.
6. Recommendation is stored and shown.
7. Terraform mapping is found.
8. Terraform plan contains only expected changes.
9. Approval is required.
10. Apply changes the intended resource.
11. Post-change verification runs.
12. Audit record contains before/after evidence.